SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA5117 - Cryptography and Network Security

ASSIGNMENT 2

QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Scenario — Secure Healthcare Data Exchange

A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration.

The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.
- Support large files without performing expensive public-key encryption on every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge

During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Problem Context – Protecting Clinical Records

The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.

Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.
- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.

Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Problem Context – RSA Key Generation

RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.

Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition gcd(e, φ(n)) = 1.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Problem Context – Online Banking Message Protection

A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.

Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.
- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-

exponent behaviour, boundary messages and invalid parameter choices.

- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry

Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.

Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Problem Context – Secure Link Between Remote Clinics

Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.

Your Challenge

- Derive the Diffie–Hellman procedure from (g^a)^b ≡ (g^b)^a (mod p).
- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.
- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk.
Evidence Expected: Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

- Identify the operation or missing security property responsible for the limitation.
- Explain how its cost or risk changes with input/key size or repeated use.
- Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design
- Experimental results
- Predictions
- Security analysis
- Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

ASSIGNMENT 2 – ANSWERS

CSA5117 – Cryptography and Network Security

Name : Dharshavardhan , Reg : 192420039

Q1. Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Symmetric encryption uses the same secret key for encryption and decryption. It is suitable for clinical records because it is fast and can process large files.

Algorithm	Structure and key features	Assessment
DES	16-round Feistel block cipher; 64-bit block; 56-bit effective key.	Insecure because its key can be brute-forced; legacy only.
3DES	Applies DES three times; 64-bit block; commonly uses 2 or 3 keys.	Stronger than DES but slow and affected by small block-size limitations; legacy.
Blowfish	16-round Feistel cipher; 64-bit block; variable key up to 448 bits.	Fast in software, but 64-bit block size makes it unsuitable for large modern data volumes.
RC5	Parameterised block cipher; typically 32/64/128-bit word sizes, variable rounds and key length.	Flexible and efficient, but security depends on selected parameters and implementation.

Block ciphers encrypt fixed-size blocks and require a mode for longer messages. Stream ciphers generate a keystream and combine it with plaintext, making them useful for continuous data streams. A stream cipher must never reuse the same key/nonce pair.

Mode	Operation and security properties
ECB	Encrypts blocks independently. Repeated plaintext blocks produce repeated ciphertext patterns; unsuitable for confidential records.
CBC	XORs each plaintext block with the previous ciphertext block. Requires a fresh unpredictable IV; encryption is sequential and an error affects the current and following block.
CFB	Uses block-cipher output as a self-synchronising stream. Requires a unique IV; errors affect a limited following segment.
OFB	Generates a keystream independent of plaintext. IV must never repeat with the same key; bit errors affect corresponding plaintext bits only.
CTR	Encrypts incrementing nonce/counter values to form a keystream. Supports parallelism and random access; nonce/counter reuse is catastrophic.

ECB reveals patterns because identical blocks have identical ciphertext. Reusing an IV/nonce can repeat keystreams or expose relationships between messages, depending on the mode. For a new deployment, use authenticated AES-GCM (or ChaCha20-Poly1305) rather than the legacy algorithms above. DES ranks last; 3DES is legacy and slow; Blowfish is limited by its 64-bit block; RC5 is flexible but less standardised. The practical recommendation is modern authenticated encryption, with RC5/Blowfish/3DES/DES retained only for controlled compatibility testing.

Implementation evidence: the student should encrypt and decrypt the same test message using at least three algorithms, then verify that the recovered plaintext equals the original. Compare execution time for increasing input sizes using identical test data and report measured values; no runtime measurements are claimed in this written answer.

Q2. Number-Theory Engine and Prime Selection for RSA (6 Marks)

The greatest common divisor (GCD) is found using Euclid's algorithm: repeatedly replace (a,b) with (b,a mod b) until b = 0. The Extended Euclidean Algorithm also finds x and y such that ax + by = gcd(a,b). If gcd(e, φ(n)) = 1, the modular inverse $d = e^{-1} \bmod \varphi(n)$ exists.

RSA uses n = pq and φ(n) = (p−1)(q−1), where p and q are distinct primes. Select e such that $1 < e < \varphi(n)$ and gcd(e,φ(n)) = 1; calculate d such that $ed \equiv 1 \pmod{\varphi(n)}$. Fast modular exponentiation uses square-and-multiply and requires O(log e) modular multiplications, unlike repeated multiplication.

Method	Strength / limitation
Trial division	Simple and deterministic for small candidates; slow for large integers.
Fermat test	Fast probable-prime test, but some composites can pass; Carmichael numbers expose weaknesses of some Fermat-style checks.
Miller–Rabin	Efficient probabilistic test; repeated independent rounds make false acceptance very unlikely.

Carmichael numbers 561, 1105 and 1729 are composite numbers that can fool basic Fermat tests for suitable bases. Trial division will identify them as composite, while Miller–Rabin is designed to detect such composites with high probability. Prime density near a large number is approximately 1/ln(N), so the expected number of random odd candidates tested rises roughly with the logarithm of candidate size.

Toy example: p = 11, q = 13 gives n = 143 and φ(n) = 120. Choose e = 7, since gcd(7,120)=1. The inverse is d = 103 because $7 \times 103 = 721 \equiv 1 \pmod{120}$. This illustrates the arithmetic only; production RSA needs cryptographically generated large primes, secure randomness and approved key sizes. 1024-bit RSA is no longer recommended; 2048-bit is a common minimum baseline, while 3072-bit provides a longer classical security margin. Operation counts and timing should be measured in code; small classroom timings cannot predict large-key performance exactly.

Q3. RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

RSA key generation chooses primes p and q, computes $n = pq$ and $\varphi(n)$, selects e coprime to $\varphi(n)$, and calculates $d = e^{-1} \bmod \varphi(n)$. Encryption is $C = M^e \bmod n$ and decryption is $M = C^d \bmod n$, for an encoded message integer M satisfying $0 \leq M < n$.

Hand-checkable example: p=5, q=11, n=55, $\varphi(n)=40$, e=3 and d=27 because $3 \times 27 = 81 \equiv 1 \pmod{40}$. For M=7, $C = 7^3 \bmod 55 = 13$. Decryption gives $13^{27} \bmod 55 = 7$, recovering the message.

- Textbook RSA is deterministic: the same message and key produce the same ciphertext, revealing equality and enabling chosen-plaintext and structural attacks.
- Small messages or poorly selected small exponents can be vulnerable to mathematical attacks, especially when padding is absent or the same message is sent to several recipients.
- Messages at or above n are invalid; weak or invalid parameters, repeated primes, poor randomness and weak key generation can undermine even a large modulus.
- If an attacker factors n into p and q, the attacker computes $\varphi(n)$ and then obtains d using the modular inverse.

OAEP (Optimal Asymmetric Encryption Padding) adds randomized encoding before RSA encryption. It prevents textbook RSA's deterministic structure and is the recommended padding for RSA encryption. RSA should encrypt only small key material, not large clinical files. Classroom factoring of small moduli by trial division or Fermat's method demonstrates principles only; it does not estimate the difficulty of factoring properly generated 2048- or 3072-bit moduli.

Q4. Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Diffie–Hellman allows two parties to derive a common secret over a public channel. Public parameters are prime p and generator g. Clinic A selects private a and sends $A = g^a \bmod p$. Clinic B selects private b and sends $B = g^b \bmod p$. A calculates $B^a \bmod p$ and B calculates $A^b \bmod p$. Both obtain $g^{ab} \bmod p$.

Step	Clinic A	Clinic B
Private value	Choose a=6	Choose b=15
Public value	$A = 5^6 \bmod 23 = 8$	$B = 5^{15} \bmod 23 = 19$
Shared secret	$19^6 \bmod 23 = 2$	$8^{15} \bmod 23 = 2$

Unauthenticated DH is vulnerable to a man-in-the-middle (MITM): an attacker intercepts each public value and establishes one separate secret with each clinic, allowing interception and modification.

Digital signatures over the exchange transcript, verified using trusted certificates, authenticate the public values and prevent this attack.

Key-management policy: generate keys using a cryptographically secure random generator; store private keys in protected hardware or encrypted key stores; restrict access; rotate keys periodically and after suspected compromise; revoke certificates promptly; maintain audit logs; derive separate session keys for encryption and authentication; use short-lived session keys; and re-establish sessions after compromise. DH private values must be ephemeral and securely erased after use. Use validated groups/parameters and a secure key-derivation function (KDF). Brute-force discrete-log tests on toy groups grow rapidly but do not represent the security of approved production groups.

Q5. ECC and Public-Key Strategy Comparison (5 Marks)

Elliptic-curve cryptography (ECC) uses points on an elliptic curve over a finite field. A private key is a random integer d and the public key is Q=dG, where G is a defined base point. ECDH derives a shared secret by computing dA QB = dB QA.

Toy example over the curve $y^2 = x^3 + 2x + 2 \pmod{17}$, with base point G=(5,1) of order 19: let private keys be dA=5 and dB=7. Their public keys are QA=5G and QB=7G. Both compute the same shared point: 5QB=35G=16G and 7QA=35G=16G (modulo the point order 19). This illustrates agreement; production systems must use standard curves and validated libraries.

Feature	RSA	Finite-field DH	ECC / ECDH
Typical public-key size for comparable classical security	Larger (e.g., 3072-bit RSA)	Large group parameters	Smaller (e.g., 256-bit curve)
Main use	Encryption/signatures	Key agreement	Key agreement/signatures
Device impact	Higher bandwidth and often more computation	Parameter and computation overhead	Good bandwidth and memory efficiency
Deployment concern	Padding and secure prime generation	Authentication required; group validation	Secure randomness, curve validation and private-key protection

ECC provides comparable classical security with much smaller keys, reducing bandwidth, storage and communication overhead. For constrained clinic devices, use authenticated ECDH (such as ephemeral ECDH authenticated by certificates/signatures) to derive a session key, then use AES-GCM for data. Ensure secure random-number generation, parameter validation, side-channel-resistant implementations and protected private keys.

Q6. Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

For a digital signature, the sender computes a cryptographic hash of the document and signs the hash with the private signing key. The receiver hashes the received document and verifies the signature using the sender's public key. A valid signature supports integrity, sender authentication and non-repudiation when keys and identity management are trustworthy.

Scheme	Key / operation characteristics
RSA-PSS	RSA-based signature; signing uses private key and verification uses public key; PSS uses randomized padding.
DSA/DSS	Discrete-log signature; requires a unique, unpredictable per-message nonce. Reuse or leakage of the nonce can reveal the private key.
ECDSA	Elliptic-curve signature with compact keys/signatures; nonce must be unique and secret, or generated deterministically by a secure standard.

Hybrid encryption is suitable for large healthcare files: (1) generate a fresh random symmetric session key; (2) encrypt the file with AES-256-GCM and a unique nonce; (3) establish or protect the session key using authenticated ECDH or RSA-OAEP; (4) sign the file metadata/hash and key-exchange transcript; (5) send ciphertext, nonce, wrapped/derived key information, certificate and signature. The receiver authenticates the sender, verifies the signature, obtains the session key and decrypts the file.

Tampering with ciphertext or the authentication tag causes authenticated decryption to fail. Changing the signed file or signature causes signature verification to fail. Altering the protected session key causes key recovery or authentication to fail. Direct RSA encryption is inefficient and cannot encrypt arbitrary large files; hybrid encryption performs symmetric encryption over the payload and only a small public-key operation for key establishment.

Quantum key distribution (QKD) uses quantum states to establish keys; measurement by an eavesdropper disturbs the states and can be detected through an increased error rate. A sufficiently capable fault-tolerant quantum computer running Shor's algorithm threatens RSA and ECC. The network should inventory cryptographic dependencies, use crypto-agile designs, follow standards-based post-quantum key-establishment and signature migration, test hybrid classical/post-quantum handshakes, and plan certificate/key replacement. QKD does not replace authentication, endpoint security, key storage, access control or ordinary key management.

Q7. Diagnose, Redesign and Validate the Complete Strategy (11 Marks)

A. Diagnose the Bottleneck or Weakness (2 Marks)

The main bottleneck is using public-key cryptography directly on every block of a large file. Public-key operations are computationally expensive and increase processing time as the number of blocks and remote clinics grows. A second weakness is unauthenticated DH, which permits MITM attacks. ECB pattern leakage and nonce reuse are additional risks.

B. Define the Design Objective (1 Mark)

Objective: encrypt each clinical file using one fresh symmetric session key, limit public-key work to authenticated key establishment, and ensure that any unauthorised modification is detected before records are accepted.

C. Design the Revised Strategy (3 Marks)

Layer	Revised design	Benefit
Bulk data	AES-256-GCM with a unique nonce per key; chunk very large files with safe unique nonce allocation and authenticated metadata.	Fast encryption, confidentiality and integrity; avoids ECB pattern leakage.
Key establishment	Ephemeral ECDH with certificate/signature authentication and a standard KDF.	Small key material and forward secrecy when ephemeral keys are securely erased.
Identity and integrity	Digital signatures on exchange transcript and relevant document metadata; certificate validation.	Authenticates clinics and prevents MITM or undetected alteration.
Key management	Secure key generation/storage, access control, rotation, revocation, short session lifetime and incident recovery.	Limits exposure and supports large multi-site deployment.

The revised design uses symmetric encryption for bulk data and public-key cryptography only to establish/authenticate session keys. It requires certificates, ephemeral private values, session-key state and nonce tracking. Trade-offs include certificate lifecycle complexity, secure device provisioning and the need to protect endpoints and signing keys.

D. Integrate Quantum Readiness (1 Mark)

RSA and ECC key establishment/signatures are vulnerable to future large-scale quantum attacks. Maintain a cryptographic inventory and migrate through standards-based post-quantum algorithms, using hybrid classical/post-quantum key establishment during transition where appropriate. QKD may help distribute keys on specialised links but does not replace endpoint authentication, signatures, secure storage or key-management processes.

E. Predict, Implement and Validate (2 Marks)

Prediction: hybrid encryption will reduce public-key operations from one or more per data block to a small, fixed number per session/file, while authenticated encryption will detect tampering. Validate with the same test file sizes under both designs. Record public-key operation counts, symmetric operation counts, execution time, ciphertext size, successful decryption, signature verification and tamper-detection outcomes.

Validation test	Expected result
Encrypt/decrypt original file	Recovered file is byte-for-byte identical.
Modify one ciphertext bit or tag	AES-GCM authentication fails; plaintext is not accepted.
Modify signed metadata or signature	Signature verification fails.
Attempt unauthenticated DH MITM	Attack succeeds in the baseline; authenticated signatures/certificates reject substituted values.
Increase file size	Symmetric processing scales with file size; public-key operation count remains approximately constant per session/file.

No fabricated execution times or measured scores are reported here. The above are predicted outcomes; actual timings and operation counts must be filled from the student's own implementation and test runs.

F. Defend the Final Design (2 Marks)

The final architecture is preferred because it combines efficient authenticated symmetric encryption for large files with authenticated ECDH for session-key establishment and digital signatures for identity and integrity. It avoids DES, 3DES, ECB and textbook RSA, reduces public-key workload, and is suitable for constrained clinics. Residual risks include compromised endpoints, stolen private keys, certificate errors, implementation flaws, side-channel leakage, nonce misuse and future quantum attacks. These are reduced through device hardening, hardware-backed keys, monitoring, rotation/revocation, secure implementation and a planned post-quantum migration.

Conclusion

A secure multi-site healthcare network should use authenticated hybrid encryption: AES-256-GCM for clinical data, authenticated ephemeral ECDH for session-key establishment, digital signatures and certificates for authentication, and disciplined key management. The design provides efficient large-file transfer while supporting integrity, confidentiality, scalability and future cryptographic migration.